A Report on The Conceptual Architecture of WezTerm

2026-10-09

By: Ethan Rose [23nfd5@queensu.ca], Jonah Schwartz [jonah.schwartz@queensu.ca], Emilia Markovic [23jf44@queensu.ca], Addison Lovegrove [23ppb3@queensu.ca], Emili Ribakov [emili.ribakov@queensu.ca], Alyssa Pascal [26ln26@queensu.ca]

Abstract

This report presents the conceptual architecture of WezTerm, a cross-platform terminal emulator and Multiplexer. The system is organized around several major components: Window /Graphical User Interface (GUI), Multiplexer (mux), PTY and Terminal Emulation(Pane), Renderer, Font discovery and shaping, Command-line Interface(CLI), and Lua-based configuration. These components cooperate to manage local and remote terminal sessions, process user input and program output, and maintain terminal state. The most significant architectural decision is the separation of the GUI from the Multiplexer: terminal sessions are owned by the Multiplexer, so sessions can exist independently of a GUI and remote sessions can be displayed as local ones.WezTerm's architecture is heterogeneous rather than based on a single architectural style. In the sense of Garlan and Shaw, WezTerm exhibits layered characteristics in the separation between its User Interface, Core Multiplexing and System and External layers. Its local and remote multiplexing follows a client-server style. Control flow is event-driven; the Multiplexer routes input to the active terminal session, and new output from a session notifies the GUI, which schedules the redraw. The architecture also supports concurrency through independent pane processing, GUI event handling, and non-blocking network I/O. The report illustrates these relationships through use cases involving remote sessions and Lua-based configuration, showing how the major components collaborate in practical scenarios.

1 Introduction and Overview

This report presents the conceptual architecture of WezTerm. It is intended to help a developer understand how the system is divided into major components, how those components interact, and why the architecture is structured the way it is. The design was derived from WezTerm's official documentation, supported by maintainer and user discussions. The analysis focuses on high-level components, architectural styles, control and data flow, concurrency, performance-critical areas, and support for future change.

Historically, terminals were hardware devices used to enter and display data on a computer. On Unix systems, they passed byte streams, flow control, and signals between the user and the computer through the kernel's teletypewriter (TTY) subsystem. A shell program, such as zsh or bash, interprets the user's input, produces output, and launches other programs, communicating with the terminal through this subsystem. Modern systems replace the physical device with software. Terminal emulators such as WezTerm create a pseudo terminal (PTY), which has a controller and client side[5],[6]. The terminal emulator holds the controller side and carries data and window-size information, while the client side is the plain I/O stream handed to the shell. The Terminal Emulator then reads the program's output, interprets the escape sequences to produce a display,

and encodes the user's keyboard/mouse input as byte sequences to send back to the program. In this way, it emulates a classic hardware terminal in software.

WezTerm is a free, open-source, cross-platform terminal emulator created by Wez Furlong and supported by community contributions and sponsorship. As a product, it combines two tools that users often run separately, a terminal emulator and a terminal Multiplexer, and aims to provide a more capable and adaptable experience, giving users one terminal that behaves the same across operating systems, works with remote machines as naturally as local ones, renders text quickly on the GPU, and can be extensively customized through Lua. Compared with more minimal terminals, WezTerm trades speed for richer features and higher quality.

The product goals explain much of WezTerm's architecture, which consists of seven major components. The most significant design feature is separating the GUI from the Multiplexer, with a layered and client-server style that works together to support this separation. Terminal sessions are owned by the Multiplexer rather than the window displaying them, so a session can exist without a window, and remote sessions can be displayed exactly like local ones. To stay fast, rendering, the most performance-critical part of the system, is done on the GPU, although some costs remain outside WezTerm's control. The architecture supports future change mainly through components that can be swapped or extended independently, while its main limits are its dependence on platform-specific behaviour.

In this report, section 2 presents the conceptual architecture: its components, architectural styles, control and data flow, concurrency, and support for future change. Section 3 lists the information WezTerm exchanges with users, programs, files, the network and the operating system, and section 4 illustrates how the components work together through use cases. The report closes with a data dictionary, naming conventions, conclusions, lessons learned, references, and a report on the group's collaboration with AI.

2 Conceptual Architecture:

2.1 Architecture Overview

WezTerm is written in Rust and runs on Linux, macOS, Windows 10, FreeBSD and NetBSD. It consists of a Multiplexer, PTY layer, Window/GUI Layer, Renderer (GPU and CPU), CLI tool, font discovery/shaping system, and configuration through a Lua engine that work together to interpret and produce WezTerm's output and functionality [4].

The User Interface Layer holds what users see and script: the Window/GUI (with its Renderer and Font Discovery and Shaping), the Lua Config and the CLI Tool. The Core Multiplexing Layer holds the Multiplexer, which manages domains (sets of windows, tabs and panes), and the PTY and Terminal Emulation (Pane) component, which is a self-contained terminal session [2]. The System and External Layer contains what WezTerm relies on but doesn't implement: the operating system's PTY and shell, the font and GPU libraries (HarfBuzz, OpenGL, WebGPU), and remote Multiplexers reached over SSH, TLS or Unix sockets [2], [5]. Platform-specific behaviour stays here, so performance characteristics such as the Windows PTY layer are outside WezTerm's control (see 2.2.2).

WezTerm's structure is designed to support its quality and performance goals. The major decision to separate the GUI from the Multiplexer lets remote panes behave like local ones, and the Renderer and Lua configuration can be swapped or extended without touching the Multiplexer (see

2.5) [2]. The layering is not strict, because the CLI and GUI both act as clients of the Multiplexer, and Lua configuration defines the Multiplexer's domain definitions. Section 2.2 describes each component in detail, and section 2.3 discusses this in terms of architectural styles.

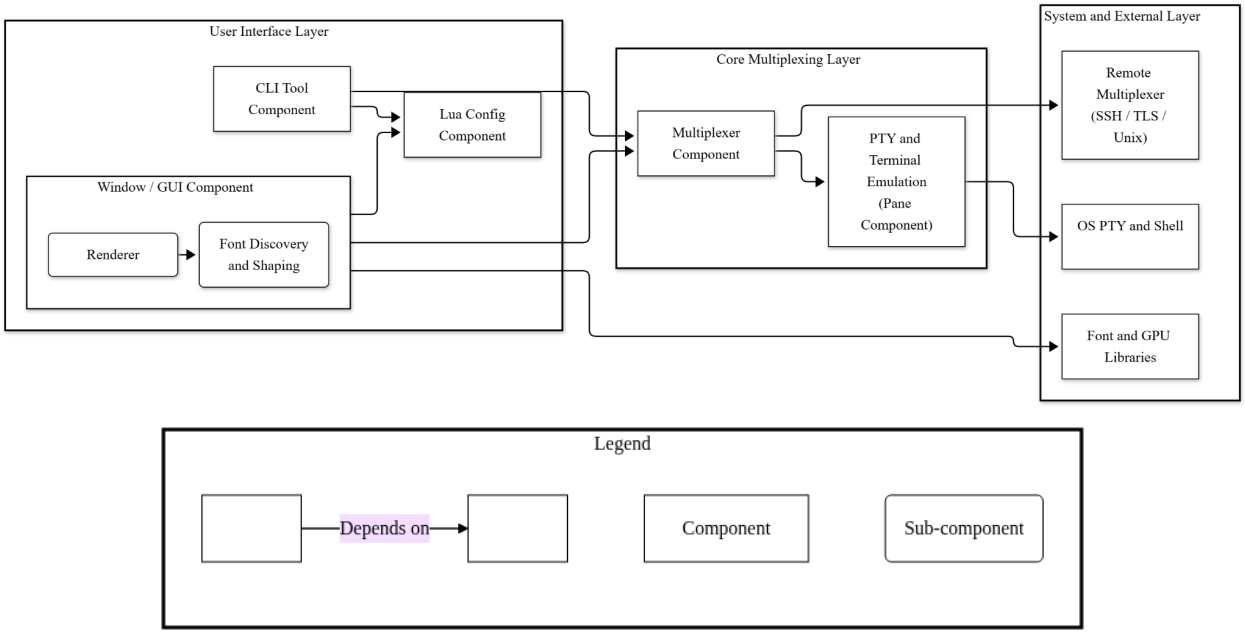


Figure 1: WezTerm high-level conceptual architecture.

2.2 Components

2.2.1 Multiplexer

A terminal Multiplexer is a software program that lets you run, manage, and switch between multiple terminal environments inside a single terminal window. WezTerm lets you multiplex local tabs and windows, which persist until closed. In WezTerm, multiplexing is based on domains: a domain is a distinct set of windows, tabs, and panes. On startup, WezTerm creates a default local domain to connect to and manage the windows and tabs in the UI, and you can configure additional domains to start, such as SSH domains, Unix domains, and TLS domains. An SSH domain connects to a WezTerm Multiplexer on a remote host over SSH. A TLS domain connects over an encrypted TCP connection. A Unix domain connects to a Multiplexer over a Unix socket and can also be used to connect into WSL[2].

When the GUI or a mux server starts, WezTerm emits startup events that the Lua configuration can respond to, for example by creating an initial set of windows and panes. A user can also attach the GUI to an existing session on another machine or in the background using WezTerm connect [3], [12] [17].

WezTerm uses domains to create a layer of separation from the PTY to the GUI. A domain owns the windows, tabs, and panes, while the local GUI attaches to a domain to display it. The Multiplexer is controlled through a Lua API that exposes domains, windows, tabs, and panes, and a CLI with subcommands for spawning, splitting, listing, and sending text to panes. Whether the domain is a remote SSH or TLS domain or a Unix domain over a local socket, the Multiplexer's goal is to manage the pane, and each pane is backed by a pseudo-terminal [2].

If the measured round-trip latency between the WezTerm client and the server exceeds the specified threshold, the client will attempt to predict the server's response to key events and echo the result of that prediction locally without waiting, hence hiding latency to the user. Users have reported that scrolling in Neovim was very slow over a multiplexed connection [23].

2.2.2 PTY and Terminal Emulation

A pseudo-terminal (PTY) is the interface between a terminal emulator like WezTerm and the program running inside it, typically a shell. WezTerm manages each PTY through the Pane component, which tracks the pseudo-terminal, its associated processes, and the parsed screen and scrollback. Panes are created and owned by the Multiplexer, and the GUI refers to them rather than owning them.

The terminal emulation part of the Pane reads the program's output from the PTY and interprets the escape sequences in it. WezTerm treats this output as a UTF-8 byte stream, decodes it into graphemes (the smallest human-perceivable characters), and applies the printable text and control sequences to the pane's terminal state. That state holds the grid of cells, the cursor position, the colour and style attributes, and the scrollback, and it is updated again when the window is resized. The Renderer then reads the changed cells from this grid for display [20]. In the other direction, the Pane encodes keyboard and mouse actions as the byte sequences a terminal would send and writes them to the PTY.

The operating system's PTY layer sits outside WezTerm's control. When a user found that printing 100,000 numbers took up to 45 seconds in WezTerm on Windows, the maintainer found Windows Terminal was even slower, showing that the bottleneck was the Windows PTY layer both rely on rather than WezTerm itself. The suggested workaround, running the shell in WSL through a multiplexing domain, shows that the architecture handles external bottlenecks through flexibility rather than optimization: because domains are interchangeable, a slow PTY can be bypassed without changing the rest of the system.

2.2.3 Renderer - Front-end GPU/CPU rendering

The Renderer takes the parsed grid of cells (terminal state) and converts it into pixels on screen. Its key requirement is speed, as every keystroke and line of output must be drawn within a frame budget of about 16 ms at the default 60 fps [1], or the user sees lag. This makes it the most performance-critical part of WezTerm.

The Renderer reads the changed cells for the Terminal emulator(pane)'s terminal state and requests matching glyphs from the font system(see 2.2.4). Each rasterized glyph is stored once in a GPU glyph atlas, which works like a set of print press stamps. When the same character appears again, the GPU reuses the copy it has stored, like a stamp, and doesn't need to regenerate it. Then it stamps all the stored glyphs into their correct position to build the frame.The finished frame is shown in the window managed by the GUI, while styling such as colours and fonts comes from the Lua Config.

WezTerm provides three ways for graphics processing that can be swapped. 'OpenGL' (the default) and 'WebGPU' use the GPU while 'Software' rasterizes on the CPU[18]. This improves robustness, as WezTerm can fall back to Software when GPU drivers misbehave and selects it automatically in Windows Remote Desktop sessions, at the cost of speed [18]. WebGPU also prefers a low-power integrated GPU by default, trading speed for energy efficiency [19]. Changes to this component are risky. The default was switched to WebGPU in January 2024 and reverted

to OpenGL in the very next release [18].

2.2.4 Font Discovery & Shaping System

The Font discovery and shaping system discovers the right font for every character, shaping that into a glyph and rasterizing it into pixels that can be stored in the glyph atlas for use by the renderer. Users can modify the styling of the font and graphics that appear on screen[9]. The system should find a font for every character, falling back through several sources, and can warn the user when none is found.

The system takes the terminal state via the renderer and processes the text through three stages. First, discovery takes each character and matches it within the user's configured fonts; if it is not present, then WezTerm checks its default set and then the system's set and can warn users if no match is found[8]. Next, shaping uses the HarfBuzz library to convert characters into glyphs, merging any text ligature into single glyphs. Then each glyph is rasterized into pixels that is then passed to the renderer to be stored in the glyph atlas [7].

The font system interacts with the renderer for input and output, the shaping engine (HarfBuzz) used to create glyphs for text ligatures, Lua configuration where the user defines their specific requirements for font, and the OS system where system fonts are located. Delegating shaping to HarfBuzz makes WezTerm more evolvable, as the system benefits from HarfBuzz improvements and gets a mature shaping engine without need to develop one. However, they also then inherit any bugs and issues that they don't have control over fixing.Text quality also costs speed. One user found WezTerm noticeably slower than Alacritty and Kitty when many characters changed at once, and attributed this to its richer font rendering [10].

2.2.5 Window/Graphical User Interface

The Window/Graphical User Interface (GUI) component is what users see and interact with. It manages the graphical terminal windows, receives keyboard, mouse, and window events from the operating system, and translates interactions into actions. It also manages the graphical elements surrounding the terminal content, such as the tab bar, scrollbar, padding, and overlays, while the Renderer is responsible for rendering the terminal content itself. Its goals are to stay responsive, behave consistently across operating systems, and display remote and local panes identically.

A key architectural distinction is that the GUI does not own the terminal sessions. The mux remains responsible for domains, windows, tabs, and panes, while the GUI attaches to this state to display it[2]. Because the GUI and mux are decoupled, mux state can exist without an attached GUI.

When the GUI has to display output from the program, it flows through the PTY into the pane's terminal state, and the pane notifies the GUI. The GUI then asks the Renderer, which reads that state and uses the font system, to draw the current frame, along with the chrome and overlays. The GUI never parses program output itself.

The GUI decides when frames are drawn, while the Renderer determines how expensive each frame is. WezTerm limits how many frames per second it will attempt to draw through the max_fps setting, which defaults to 60 and can be changed in the configuration. Having this limit shows that each frame costs real CPU/GPU work, and drawing more than the screen can display wastes resources [1]. On Wayland, it is ignored and instead uses information from the compositor to schedule painting frames[1]. This shows how timing depends on the platform. Frame timing isn't fully under WezTerm's control, and has to fit around the operating environment.

2.2.6 Lua Configuration

WezTerm uses Lua 5.4 as a configuration language. The primary functions are provided by the imported WezTerm module, which exposes a wide variety of configuration options including both cosmetic customizations as well as functional features as well (font, window size, colours, key-bindings, etc.).

The Lua configuration is a running program, not just a fixed list of settings. This allows you to have logic in your configuration, including if-statements, loops, and functions. Moreover, you can split your configuration across modules, allowing for better organization, troubleshooting, as well as improved code reusability. WezTerm also dynamically refreshes while running, allowing real-time config updates.

One caveat of the Lua config comes from the fact that it is a running program. Since the config dynamically reloads, it can run many times per session. The documentation suggests avoiding starting many background processes as this can lead to increased software and OS load.

2.2.7 Command Line Interface

The CLI is the command-line entry point to WezTerm. WezTerm is deployed with two major executables: WezTerm (WezTerm.exe on Windows) for use from a terminal, and WezTerm-gui (WezTerm-gui.exe on Windows) for launching from a desktop environment. WezTerm delegates to WezTerm-gui whenever a GUI is needed. The CLI has two broad jobs: launching things and controlling a running WezTerm. Its main goals are scriptability and automation, having one consistent interface across platforms, and working with multiple running instances.

The CLI interacts with four parts of the system. It can act on the mux's windows, tabs, and panes, and attach to a Multiplexer domain. It launches the GUI by delegating to WezTerm-gui, and can target an already running GUI instance. It reads the Lua configuration and lets users override it from the command line. Externally, its main users are scripts and other tools.

Each subcommand in the CLI is independent, so new capabilities can be added as new subcommands without changing existing ones. Because the CLI only sends requests to a running GUI or mux server and holds no terminal state itself, changes inside those components, such as how panes are managed or how the screen is rendered, do not require changes to the CLI. However, the CLI depends directly on the mux's model, so major changes there would require matching changes in the CLI, and WezTerm labels this interface experimental, so user scripts may need updating as it evolves. To support testability, scripts can be used to check behaviour automatically without a person at the keyboard. Recording and replaying is useful for reproducing bugs, while commands like ls-fonts and show-keys help diagnose font and key-binding problems [4],[14].

2.3 Architectural Styles

Garlan and Shaw note that software systems commonly combine multiple architectural styles [11]. WezTerm is an example of such a heterogeneous architecture. Two of its most significant architectural styles are layered organization and client-server architecture.

Garlan and Shaw define a layered system as a hierarchy, in which each layer serves the layer above and acts as a client of the layer below, with connectors defined by the protocols between layers [11]. WezTerm exhibits characteristics of this style through its User Interface, Core Multiplexing, and System and External layers (Figure 1). The separation of these responsibilities encourages evolvability and reuse. It also allows the GUI to remain separate from the Multiplexer

[2], [3]. However, WezTerm is not strictly layered because some components interact across layers. Although this increases flexibility, changes can affect more than one nearby layer, and may require more testing.

Garlan and Shaw define client-server as a distributed-process architecture in which a server represents a process that provides services to client processes. Clients know or discover the identity of a server, while the server does not necessarily know in advance what clients will connect [11]. WezTerm's multiplexing functionality exhibits this architectural style: a Multiplexer can manage terminal sessions independently of a graphical client, and the GUI or command-line interface can connect to the Multiplexer to access those sessions [2], [3]. This is beneficial as the terminal sessions can persist independently of a particular GUI. However, the communication between the client and mux server can introduce management complexity.

An alternative architectural style would be a more event-based architecture, where the major components communicate by publishing and responding to events. This could reduce coupling, but would make debugging and testing more difficult.

2.4 Control, Data Flow, and Concurrency

Control and data move through WezTerm along two paths that meet at the Multiplexer: an input path and an output path. The system is event-driven, so components react to events instead of running in a fixed order.

On the input path, the user's keyboard or mouse input reaches the Window/GUI layer as an OS event. The GUI looks it up in the key and mouse bindings from the Lua Config and handles some actions itself, such as opening a tab. Actions meant for the terminal session go to the Multiplexer, which routes them to the focused Pane. The Pane converts the action into the bytes a terminal would send and writes them through the PTY to the shell.

The output path begins when the shell writes output and escape sequences back through the PTY, which can happen at any time, not only in response to input. The Pane's terminal emulation interprets the bytes and updates the terminal state (character grid, cursor and scrollback). The Pane then notifies the GUI that a redraw is needed, and the GUI decides when to draw, keeping within its frame-rate limit. The Renderer then reads the changed text, obtains rasterized glyphs from the Font Discovery and Shaping subsystem, caches them in the GPU glyph atlas, and presents the frame to the display through the window.

Within a process, components pass control through direct calls and event notifications. Across process or machine boundaries, control passes as messages, through the PTY to the shell and over Unix sockets, TLS or SSH to remote domains. The Lua Config and CLI sit outside this pipeline: WezTerm calls Lua handlers on events such as gui-startup and mux-startup, while CLI acts as a client that sends requests to a running GUI or mux server.

WezTerm uses several units of concurrency so that no slow task freezes the interface. The GUI event loop receives OS events and schedules drawing, with its frame rate capped by max_fps [1]. Each pane's output is handled separately from the GUI so a burst of output does not freeze the window, and remote domains use non-blocking network I/O [2][3]. The GUI, a mux server and CLI commands can also run as separate processes, so the terminal session is decoupled from the interface that shows it.

2.5 Evolution and Support for Future Changes

WezTerm supports change mainly through the way its parts are arranged. The system is a pipeline in which the Multiplexer owns the sessions, each Pane holds the terminal state, and the Renderer and Window/GUI present it. Each stage hands the next a small, well-defined piece of information, so a change at one end, such as what a pane connects to, does not have to ripple through the rest. This also lets developers work on rendering, multiplexing and configuration with limited coordination.

The arrangement also lets WezTerm adapt to its environment. Platform-dependent parts, such as the window loop, the PTY and the graphics back end, sit largely at the edges of the pipeline, while multiplexing and terminal state stay the same. The Windows PTY problem was handled by reconfiguring how the system is assembled (running the shell in WSL and connecting through a Multiplexer domain) rather than by rewriting a component.

New capabilities can often be added without changing the core. Because the configuration is a running Lua program and the CLI exposes the Multiplexer's operations, users and outside tools can build new workflows on top of the system. The cost is that these interfaces become compatibility commitments once users depend on them.

Change still carries risk. Rendering is performance-critical and tied to host hardware, as the switch to WebGPU and rollback to OpenGL showed, and WezTerm depends on things it does not control, such as the OS PTY, HarfBuzz and graphics drivers. Because control flow converges on the Multiplexer, changing how it represents sessions would affect nearly every subsystem.

2.6 Implications for Division of Developer Responsibilities

WezTerm's component boundaries map naturally onto areas of developer responsibility. Rendering and font work requires graphics and typography expertise. Terminal emulation requires detailed knowledge of escape sequences and terminal standards. The Multiplexer and its domains require networking and protocol knowledge. The GUI requires familiarity with each platform's windowing system, and the Lua configuration and CLI are user-facing interfaces whose stability affects users' configurations and scripts. Because components interact through a small number of well-defined interfaces, such as the Multiplexer's model of domains, windows, tabs and panes, developers can work within one area without needing deep knowledge of the others. However, changes to these shared interfaces must be coordinated across areas, and changes to the Lua and CLI interfaces must also consider existing users.

3 External Interfaces

WezTerm sits between the user and the programs they run, so most of the information crossing its boundary is terminal content: input travelling from the user to a program, and output travelling back to the screen. It also reads configuration and fonts from files, and communicates over the network for remote sessions and maintenance tasks.

WezTerm receives keyboard and mouse input from the user and presents the rendered terminal display in return. From the programs running in each pane, it receives a UTF-8 text stream that carries formatting and control instructions along with the text itself. In the other direction, WezTerm sends the user's keystrokes and pasted text to the program in the active pane [20]. It reads its Lua configuration file and any modules it loads, and watches the configuration file so that

changes are applied automatically while it runs. [13] It also reads font files, which it shapes into glyphs using HarfBuzz. When connected to a remote Multiplexer, WezTerm sends the user's input and requests, and receives the terminal content of the remote panes [2]. Plain SSH sessions work similarly, sending keystrokes and commands to the remote shell and receiving its output. WezTerm also downloads plugins from Git repositories, and once every 24 hours by default it checks GitHub for a new stable version and notifies the user. [21] [22] From the windowing system, WezTerm receives window events such as resizing, focus changes and the window being closed, and on Wayland the compositor also tells it when to draw [1]. In return, it sends finished frames to the window. When connected to a serial port, it sends what the user types to the device and displays the data the device sends back. Through the CLI, scripts send WezTerm commands and receive results such as a list of panes or a pane's text content (see Section 2.2.7)[4].

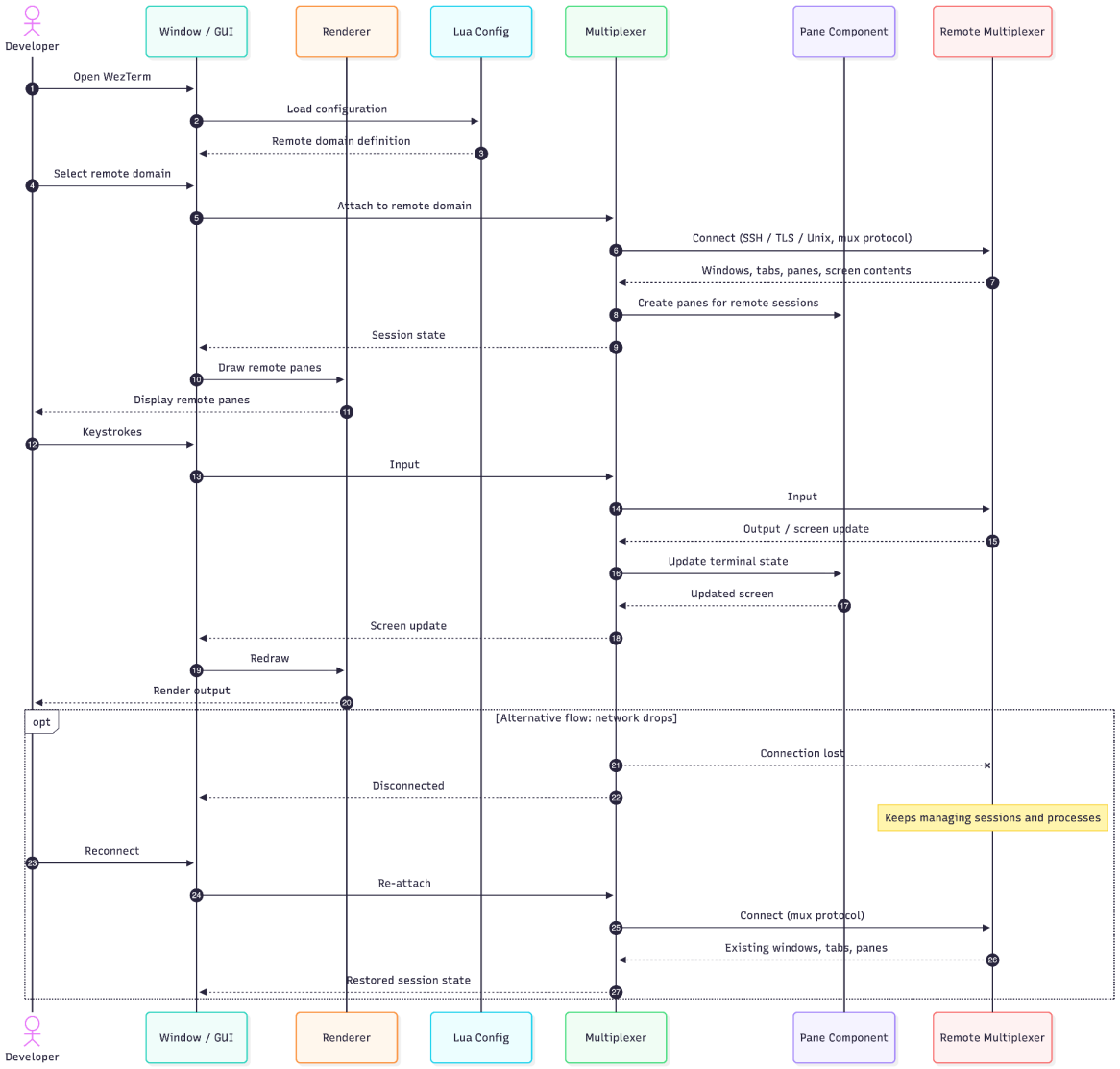
4 Use Cases:

4.1 Use Case 1

Use Case: Resume a Remote Development Session
Primary Actor: Developer

A developer who wants to resume a previously running remote terminal session while a WezTerm Multiplexer is still running on the remote machine first opens WezTerm. The Window/GUI Component loads the configuration through the Lua Config Component that defines the remote multiplexing domain. The developer then selects that remote domain, and the Multiplexer Component initiates the connection. It reaches the Remote Multiplexer and communicates with it using the mux protocol. The Remote Multiplexer sends back the current state of its windows, tabs, and panes, including the screen contents of each pane. The Multiplexer Component creates the corresponding panes and passes this state to the GUI, whose Renderer displays the remote terminal sessions locally, allowing the developer to continue working with the same remote processes. From then on, keystrokes go from the GUI through the Multiplexer component to the Remote Multiplexer, and output flows back the same way, updating the pane's terminal state before the Renderer displays the result.

If the network connection drops, the Multiplexer Component loses its connection to the Remote Multiplexer, and the GUI shows a disconnected domain. The Remote Multiplexer continues managing the terminal sessions and associated processes. When the developer reconnects, WezTerm reattaches to the existing windows, tabs, and panes, allowing work to continue without restarting the remote processes.



4.2 Use Case 2

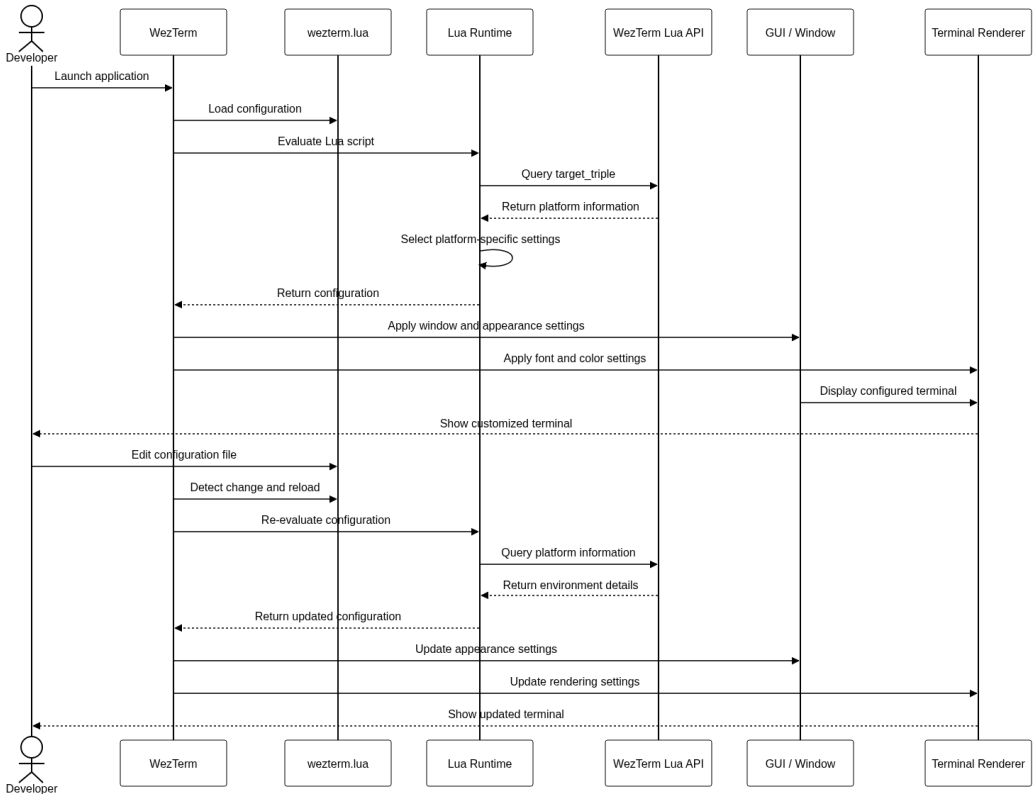
Use Case: Extensive Cross-platform Lua-based Configuration/Customization
Primary Actor: Developer

WezTerm allows developers to extensively configure their WezTerm experience through a Lua configuration, a program that runs the configuration rather than a static text file [13], [14]. A programmatic configuration lets developers easily port their existing configurations between development machines, even across different operating systems. This is accomplished through a single WezTerm.lua configuration [13], [15].

When the developer launches WezTerm, it locates and evaluates the Lua configuration file. The resulting config is used to configure the running WezTerm instance [13], [15]. During config evaluation, the configuration layer communicates directly with the WezTerm API to query the underlying runtime environment [14]. This allows for OS-specific customizations, such as modifier keybinds (Ctrl vs. Cmd). The Lua configuration is then applied to the instance, rendering visual customizations such as fonts and themes through the GUI, and applying all behavioural changes

in the background [13], [14]. WezTerm supports hot-reload, meaning this configuration can be changed at any time and will re-evaluate and apply the new configuration to the running instance [16].

By leveraging a single, programmatic WezTerm.lua file, WezTerm isolates the configuration from the environment. This abstraction enables seamless configuration portability across development environments. Simply using the same WezTerm.lua file across environments will ensure an identical, fully customized experience [14]. This process is visualized in the sequence diagram below.



5 Data Dictionary:

Components

- **Window / GUI** (component): The layer users interact with. It receives keyboard, mouse and window events, manages the tab bar and scrollbar, and attaches to the Multiplexer to display its sessions.
- **Renderer** (sub-component of GUI): Converts a pane's grid of cells into pixels using the glyph atlas. Backends are OpenGL, WebGPU or CPU software.
- **Font Discovery and Shaping** (sub-component of GUI): Finds a font for each character, shapes text into glyphs with HarfBuzz, and rasterizes them for the Renderer.
- **Lua Config** (component): The user's configuration, written as a running Lua program. It defines key bindings, fonts, colours and domains, and registers event handlers.

- **CLI Tool** (component): The WezTerm command-line entry point. It launches sessions and acts as a client that controls a running GUI or mux server.
- **Multiplexer (Mux)** (component): Manages domains and owns their windows, tabs and panes. It routes input to the focused pane.
- **PTY and Terminal Emulation (Pane)** (component): One self-contained terminal session. It tracks a pseudo-terminal and its processes, interprets program output into terminal state, and encodes user input as bytes for the PTY.
- **OS PTY and Shell** (external component): The operating system's pseudo-terminal and the shell running inside it.
- **Font and GPU Libraries** (external component): HarfBuzz, system fonts, and the OpenGL and WebGPU graphics interfaces.
- **Remote Multiplexer** (external component): A WezTerm Multiplexer on another host, reached over SSH, TLS or a Unix socket.

Multiplexing terms

- **Domain** (concept): A distinct set of windows, tabs and panes. Types are local, SSH, TLS and Unix.
- **Mux server** (process): A Multiplexer running independently of any GUI, so sessions survive without a window attached.

Terminal emulation terms

- **Pseudo-terminal (PTY)**: A software replacement for a hardware terminal, with a controller side held by the emulator and a client side given to the shell.
- **Escape sequence** (data): A control sequence in program output, such as a colour change or cursor move, that the terminal model interprets as an instruction.
- **Grapheme** (data): The smallest human-perceivable character, decoded from the UTF-8 byte stream.
- **Terminal state** (data): A pane's grid of cells, cursor position, style attributes and scrollback. The Renderer reads it.
- **Scrollback** (data): Earlier output that has scrolled off screen but is kept for review and search.

Rendering and font terms

- **Glyph** (data): The drawable shape of a character or ligature in a given font.
- **Shaping** (process): Turning characters and a font match into glyphs, merging ligatures.
- **Rasterization** (process): Converting a glyph into pixels.
- **Glyph atlas** (GPU data structure): A cache of rasterized glyphs so repeated characters are reused instead of regenerated.
- **Frame budget** (concept): The time available to produce one frame, about 16 ms at 60 fps.
- **Software renderer** (backend): The CPU-based fallback used when no usable GPU is available.

Architectural styles

- **Layered** (style): A hierarchy in which each layer serves the layer above and is a client of the layer below.
- **Interpreter** (style): A virtual machine that executes instructions. In WezTerm, the terminal model executes escape sequences against terminal state.
- **Client-server** (style): A server provides services to clients that connect to it. In WezTerm, the GUI and CLI are clients of the Multiplexer.
- **Event-based implicit invocation** (style): Components announce events and registered handlers are invoked implicitly. In WezTerm, this is the Lua event system.

6 Naming Conventions:

- TTY: teletypewriter
- PTY: pseudo-terminal
- GUI: graphical user interface
- SSH: secure shell
- TCP: Transmission Control Protocol
- GPU: graphics processing unit
- CPU: central processing unit
- CLI: command-line interface
- Mux: Multiplexer
- TLS: Transport Layer Security
- OS: operating system
- UTF-8: 8-bit Unicode Transformation Format
- I/O: input/output
- API: application programming interface
- FPS: frames per second
- ms: milliseconds
- WSL: Windows Subsystem for Linux

7 Conclusions:

In conclusion, WezTerm extends itself through specific connection points such as swappable renderers, domain types, Lua configuration and the CLI. The structure focuses on linking components in a way that has clear task boundaries that allow for smooth interaction. The central structural separation of the GUI from the Multiplexer exemplifies this defined separation and works through the interaction with the other components. The layered and client-server styles organize a flexible system between components, while interpreter and implicit invocation styles are used internally to let behaviour extend and interact between components. The flexible pattern allows WezTerm to

adapt to changes in the external interfaces that it relies upon. But this flexibility was not without cost, and WezTerm has a dependence on outside libraries and systems that becomes a compatibility commitment. Future work should focus on how these dependencies can be made more robust.

8 Lessons Learnt:

While analyzing the architecture of WezTerm through its documented resources, our team gained an appreciation for the complexity and magnitude of open-source projects like WezTerm. We initially looked for documentation giving a full overview of the conceptual design, but found that none covering the full scope of WezTerm seemed to exist. This taught us a valuable lesson about large-scale projects; a conceptual architecture is rarely written down in one place, and has to be understood from the documentation, issue discussions, and observed behaviour of the system. Moving forward, we split the report into sections so everyone could learn a different facet of WezTerm and contribute to one part of the project. This strategy worked, as everyone had their specific section and could research what was needed. However, this left our overall knowledge of the architecture at different levels, as we had all researched different topics. In the future, we plan to spend more time bringing everyone in the group up to the same level of understanding before splitting up to write our own sections. This should eliminate problems that arose in this project, such as group members struggling to understand and review sections that were outside the domain they had researched.

9 References:

[1] "Max_Fps - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/lua/config/max_fps.html

[2] "Multiplexing - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/multiplexing.html

[3] "Object: MuxDomain - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/lua/MuxDomain/index.html

[4] "WezTerm - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/index.html

[5] "What Is a Terminal? - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/what-is-a-terminal.html

[6] NHI Mgmt Group, "Pseudo Terminal," NHI Management Group. Accessed: Oct. 09, 2026. [Online]. Available: https://nhimg.org/glossary/pseudo-terminal/

[7] "Font Shaping - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/font-shaping.html

[8] "WezTerm.Font_With_Fallback - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/lua/WezTerm/font_with_fallback.html

[9] "WezTerm.Font - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/lua/WezTerm/font.html

[10] expipiplus1, "Slow Performance With Large Redraws," GitHub. Accessed: Oct. 09, 2026. [Online]. Available: https://github.com/WezTerm/WezTerm/issues/4293

[11] D. Garlan and M. Shaw, "An Introduction to Software Architecture," Carnegie Mellon University, Tech. Rep. CMU-CS-94-166, Jan. 1994.

[12] "mux-startup - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. Available: https://WezTerm.org/config/lua/mux-events/mux-startup.html

[13] "Configuration - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/files.html

[14] "Lua Reference - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/lua/general.html

[15] "WezTerm.Config_Builder - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/lua/WezTerm/config_builder.html

[16] "Automatically_Reload_Config - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/lua/config/automatically_reload_config.html

[17] "gui-startup - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online] Available: https://WezTerm.org/config/lua/gui-events/gui-startup.html

[18] "front_end - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/lua/config/front_end.html

[19] W. Furlong, "WebGPU front_end," GitHub, Issue #2756. Accessed: Oct. 09, 2026. [Online]. Available: https://github.com/WezTerm/WezTerm/issues/2756

[20] "Escape Sequences - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/escape-sequences.html

[21] "Plugins - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/plugins.html

[22] "check_for_updates - Wez's Terminal Emulator," WezTerm.org. Accessed: Oct. 09, 2026. [Online]. Available: https://WezTerm.org/config/lua/config/check_for_updates.html

[23] joexue, "The Multiplexing is very slow," GitHub, Apr. 13, 2022. Accessed: Oct. 09, 2026. [Online]. Available: https://github.com/WezTerm/WezTerm/issues/1872

10 AI Collaboration Report:

Our group's primary AI teammate for this deliverable was Anthropic's Claude Opus 5.5, accessed through the claude.ai web interface in October 2026. We selected Claude as our main AI teammate because the deliverable depended heavily on interpreting several long documents at once: the

project specification, the marking rubric, course reference material on architecture documentation, and successive drafts of our own report. Claude could take all of these as uploaded files within a single conversation and refer back to them across many exchanges. This made it well suited to the task we most needed help with, which was evaluating our work against the requirements rather than generating content. Allowing each member to choose their own tool gave everyone the flexibility to work in the way that suited them best.

Tasks Assigned to the AI Teammate

1. **Requirements and rubric alignment.** We asked the AI to review drafts against the specification and marking rubric and identify missing or weak content. This was well suited to an AI because it is a systematic comparison of two documents, which is tedious to do by hand across a long report.
2. **Conceptual clarification.** We used the AI to explain architectural concepts and WezTerm components, such as the role of the Window/GUI layer and whether a given architectural style applies within or between components. These explanations supported our understanding, but we verified them against WezTerm's documentation before relying on them.
3. **Report structure and organization.** The AI suggested section ordering and headings, and advised on how to avoid overlap between the introduction and the architecture section. Structural advice carries little factual risk, and the AI could draw on the uploaded course material on documenting architectures.
4. **Consistency checking and writing support.** Because six members wrote different sections, we asked the AI to identify inconsistencies across the full draft and to provide guiding questions and feedback for the conclusions.

Rather than designating a single prompt engineer, each member acted as their own prompt engineer, using AI independently to support the sections they were responsible for. This allowed members to tailor their prompts to the specific questions raised by their own sections. Where AI feedback affected other parts of the report, such as inconsistencies between sections or suggested changes to shared terminology, the member who received it shared it with the relevant authors, and changes were agreed on before being made.

Our prompting strategies improved noticeably over the project. Prompts were further refined by narrowing their scope, for example, asking the AI to focus only on A1 when its suggestions began referring to later deliverables, and by asking targeted conceptual questions rather than open-ended ones. Members shared these lessons with each other, so the group's prompting improved as a whole.

We treated the AI as a reviewer whose suggestions required verification, not as an authority. Our quality control involved three steps. First, any factual claim about WezTerm suggested or clarified by the AI was checked against the official WezTerm documentation or the project's GitHub issues before inclusion. Where the AI itself flagged uncertainty, such as how keyboard input is encoded or whether a concurrency claim was supported by its citation, we treated the point as unverified until confirmed; this extended to any sources the AI used and claimed against. Second, every suggested change was discussed with the section's author before being adopted. Third, any AI-assisted wording was rewritten or edited by a group member for accuracy and consistency with the rest of the report.

We estimate that the AI contributed approximately 15% of the final deliverable. This breaks down as follows:

- Review and feedback that shaped revisions (about 7%). This had the largest influence but produced no text directly.
- Report structure and organization (about 4%).
- Conceptual explanations that informed our understanding (about 2%).
- AI-assisted wording that was edited by group members (about 2%).

All research, source selection, architectural analysis, diagrams and the large majority of the written content were produced by group members. The AI's influence was mainly on the quality and consistency of the report rather than on its content.

Overall, the AI saved time, particularly in reviewing a long, multi-author report against detailed requirements. It identified issues we had overlooked, including inconsistent descriptions of the system's layers across sections, duplicated content left over from earlier drafts, and required topics we had not yet covered. However, it also created new work. Its responses were often long and required consistent reading for sometimes limited value, and every factual suggestion required verification, which took real time.

The AI influenced our decision-making most by acting as an outside reviewer. Its feedback gave members concrete points to discuss, which helped us coordinate revisions across six authors. Because each member used AI independently, and with different models, some inconsistencies in terminology arose between sections, which suggests that a consolidated review of the whole report would be valuable in future deliverables.

The main lessons we will carry forward are that AI output improves greatly when it is given the requirements and context up front, that specific questions produce more useful answers than open-ended ones, and that AI is most valuable for checking and organizing work rather than producing its substance. For future deliverables, we plan to agree on shared terminology before writing, record our key prompts as we go, and verify every factual claim at the point it is introduced.